

Quantizing a Streaming VLM for Consumer Silicon

Landing a 4B streaming VLM on a consumer Mac: measured footprint, latency, packaging

2i

2026-07-29

Contents

1	Summary	3
2	Background	3
2.1	Why run a VLM at the edge	3
2.2	Model architecture	3
2.3	Numbers reported by the authors (reference only, not ours)	4
3	Method	4
3.1	Hardware and stack	4
3.2	Quantization method	4
3.3	Test input	4
3.4	Metrics	4
4	Results	4
4.1	Measured table	4
4.2	Figures: memory and throughput	5
5	Interpretation: the duality of shrinking memory and shrinking speed	6
6	Limitations and scope	6
7	Data and reproducibility	7
8	References	7

1 Summary

We measured what happens when a SOTA 4B-class streaming vision-language model (VLM) is loaded onto a consumer Apple-silicon Mac instead of a datacenter GPU. The subject is **microsoft/Mage-VL**, released under the **Apache-2.0** license, which combines a Mage-ViT visual encoder with a **Qwen3-4B-Instruct-2507** language backbone. On an Apple M4 Pro (48GB unified memory) we ran real image-understanding inference at three precisions, bf16, weight-only INT8, and weight-only INT4, and measured device memory, decode throughput, time-to-first-token, and answer quality. The result splits into two halves. First, quantization did cut device memory as expected: from 9.49GB at bf16 to 5.86GB at INT8 (-38%) and 4.15GB at INT4 (-56%), and on-disk footprint showed room to shrink from 9.8GB down to 2.9GB for a community MLX 4-bit checkpoint. Answer quality held up at every precision on our test image. Second, and this is the non-obvious finding, the same quantization moved decode speed in the opposite direction. On the **transformers** + Metal (MPS) stack, decode throughput fell from 13.01 tok/s at bf16 to 2.31 tok/s at INT8 and 0.47 tok/s at INT4, that is, quantization made decoding slower, not faster. Memory and speed are separate axes, and on a runtime without fused low-bit matmul kernels, quantization pays out in memory savings and collects its cost in decode latency. The streaming-video fast path (the DCVC-RT neural codec) is CUDA-only and was never attempted on Apple Metal; this is a scope boundary, not a model failure, and it defines the next packaging task (an MLX architecture port). 2i sells this measured packaging work, quantization choice, runtime choice, and scope definition, not the model itself.

2 Background

2.1 Why run a VLM at the edge

Demand keeps growing for putting a vision-language model wherever a camera already sits, retail-ops CCTV, patrol-robot cameras, factory inspection lines. The problem is that most of these sites have no power, space, or network budget for a datacenter GPU. Streaming video to the cloud for inference runs into latency, bandwidth, and increasingly security or sovereignty requirements that mandate on-premise processing. The question becomes simple: can a SOTA-class VLM run not in a server room but on a consumer-grade computer sitting in the back of a store.

microsoft/Mage-VL is a good test case for that question. It supports both image-text understanding and video/streaming understanding at a 4B scale, and it ships under the **Apache-2.0** license. That license is one reason this paper exists at all: it means the model can be packaged commercially and deployed for a client rather than kept as a research demo, which makes “what do you gain and give up by running this on consumer hardware” a business question, not just a curiosity.

2.2 Model architecture

Mage-VL has two parts. The visual encoder is a proprietary architecture, Mage-ViT, at 4B scale, and the language backbone is **Qwen3-4B-Instruct-2507**. The parameter count we actually measured after loading the model is **4.742B**, consistent with the public “4B” labeling. On top of these two components the model ships a “StreamMind” event gate, auxiliary weights (1.07GB) meant to proactively trigger on salient moments in a stream, and a DCVC-RT neural video codec built as C++/CUDA extensions to accelerate frame processing on the streaming path.

We flag this paper’s scope boundary up front. Understanding a single image, or an individual video frame, never touches the neural codec at all, so it requires no CUDA. The genuinely fast streaming-video path, however, is bound to DCVC-RT’s .cu CUDA kernels, and that path does not run on Apple Metal as shipped. So this paper measures the image/frame-understanding path, and treats the streaming-video path not as “broken” but as an explicitly separated future task, porting the codec, which we name rather than blur. Keeping that separation honest is this paper’s core discipline.

2.3 Numbers reported by the authors (reference only, not ours)

The Mage-VL authors report benchmarks run on an 8xB200 datacenter node: visual tokens cut by more than 75%, up to a 3.5x wall-clock speedup over uniform frame sampling, accuracy above 96.1% on Food-101 and above 86.3% on ImageNet at 676 tokens, an F1 of 16.35 and ROC-AUC of 83.14 on SoccerNet streaming, and 64.00 on OVO-Bench. Every one of these numbers is **as reported by the authors**, and none of them is compared directly against anything we measured. Our question in this paper is entirely different: what happens when this model is loaded onto a consumer Mac instead of a datacenter.

3 Method

3.1 Hardware and stack

We measured on an Apple M4 Pro, 48GB unified memory, macOS, arm64, an ordinary consumer computer (Mac-mini class), not a datacenter GPU. The software stack was `transformers` 5.12 and `torch` 2.12, with the compute device set to MPS, Apple’s Metal backend.

3.2 Quantization method

Quantization used `optimum-quanto`’s weight-only path, applied only to the language-model submodule (`language_model`). The vision encoder (Mage-ViT) stayed in bf16 across all three runs, so the only variable we changed was the precision of the language backbone’s weights (bf16 -> INT8 -> INT4), keeping the visual path fixed.

Separately, we attempted to load a community-produced INT4 checkpoint built for the MLX runtime via `mlx-vlm`. That attempt failed to load, as detailed below, but it left one useful data point, on-disk footprint, which we report.

3.3 Test input

The test image is **synthetic**, not a real photograph. We built a 768x512 “station departures board” scene ourselves precisely so that we would know the ground truth exactly. The title reads “CENTRAL STATION - DEPARTURES”, followed by three rows: “14:05 / PLATFORM 2 / ON TIME”, “14:20 / PLATFORM 5 / DELAYED”, “14:35 / PLATFORM 1 / BOARDING”, along with a train pictogram and a red STOP sign. We used a synthetic image rather than a real photo specifically so we could fully control the ground truth and judge unambiguously whether the model actually read the screen or merely produced a plausible-sounding guess. Disclosing this is part of this paper’s honesty commitment.

The prompt asked for a detailed description (“Describe this image in detail...”). Prompt length including vision tokens was 419 tokens, and generation was fixed at 128 tokens per run.

3.4 Metrics

We measured four things. **Device memory** is the memory MPS actually allocated. **Decode throughput** is generated tokens per second, averaged over the 128-token generation window. **Time-to-first-token** (TTFT) is the latency from the end of prompt processing to the first output token, capturing the cost of the 419-token prefill. **Quality** is a qualitative check of whether the model’s description accurately includes the ground-truth elements of the synthetic image (the title text, the three rows of time/platform/status, the train pictogram, the STOP sign). This is not an automated accuracy score; it is a ground-truth comparison against a single synthetic image, and we are explicit about that scope.

4 Results

4.1 Measured table

The table below covers all three precision paths plus the MLX attempt. Every number is a direct measurement on the Apple M4 Pro, except the on-disk size in the MLX row, which is the published file size of the community

checkpoint.

Precision	Runtime	Device memory (MPS alloc)	Decode tok/s	TTFT	Quality (synthetic test image)	On-disk
bf16	transformers + MPS	9.49 GB	13.01	2.94 s	Accurate (correctly read the board title and structure)	9.8 GB (weights + gate)
INT8 (quanto, weight-only, LLM tower)	transformers + MPS	5.86 GB (-38%)	2.31	1.77 s	Identical to bf16, still accurate	In-memory only
INT4 (quanto, weight-only, LLM tower)	transformers + MPS	4.15 GB (-56%)	0.47	9.91 s	Accurate (reworded, content still correct)	In-memory only
INT4 (community MLX checkpoint)	mlx-vlm	Failed to load	Not measurable	Not measurable	Not measurable	2.9 GB

The bf16 model load time was approximately 12 seconds. The MLX 4-bit community checkpoint failed to load because `mlx-vlm` mainline does not register the custom `mage_vl` architecture (“Model type `mage_vl` not supported”). We therefore could not benchmark its speed and report only its on-disk footprint, 2.9GB, as the achievable quantized size. Measuring its speed would require porting the `mage_vl` architecture into `mlx-vlm` directly.

4.2 Figures: memory and throughput

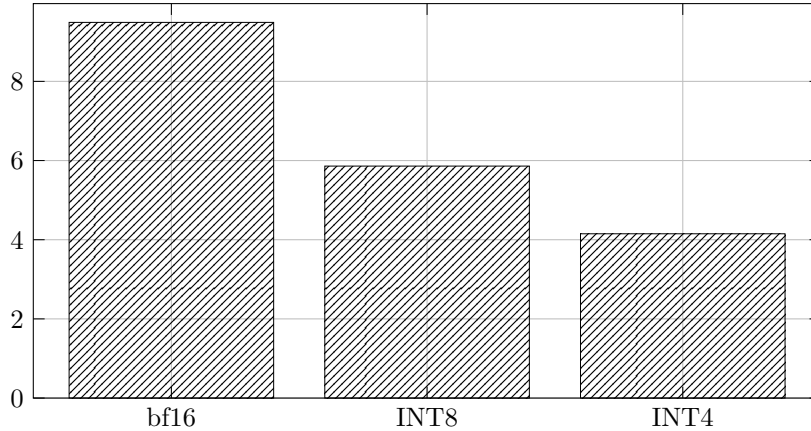


Figure 1: Device memory by precision

Read the two figures together and this paper’s core point is visible at a glance. Memory falls monotonically as precision drops, and throughput also falls monotonically as precision drops. The two curves do not move together, they move in opposite directions relative to what one would naively expect, and that divergence is what section 4 explains.

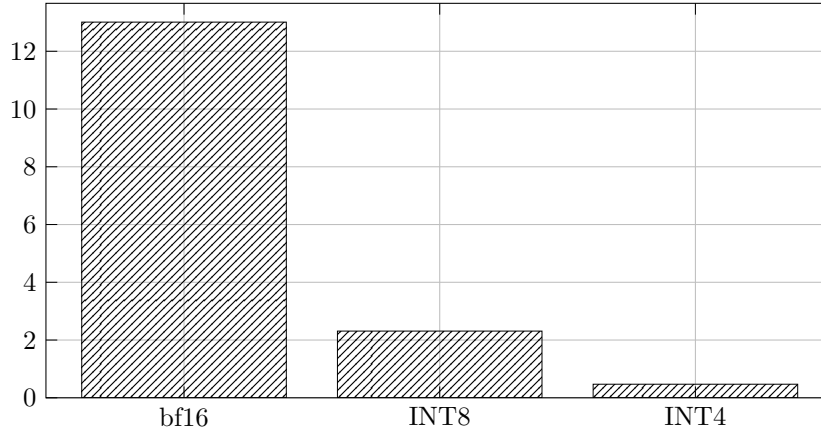


Figure 2: Decode throughput by precision

5 Interpretation: the duality of shrinking memory and shrinking speed

The standard expectation is that quantization makes a model both smaller and faster. This measurement confirms half of that expectation and directly contradicts the other half. Memory savings behaved as expected: device memory dropped 56%, from 9.49GB at bf16 to 4.15GB at INT4. On the same 48GB unified-memory Mac, that frees headroom for other work (a longer context window, or another process running alongside), and in principle it means this model could fit on hardware with less memory to begin with.

Decode throughput moved the opposite way. It fell 82%, from 13.01 tok/s at bf16 to 2.31 tok/s at INT8, and 96% relative to bf16 at INT4’s 0.47 tok/s. Two metrics that should move together diverged, and the reason is not the hardware, it is the **kernel**. `optimum-quanto`’s weight-only quantization stores weights at low bit-width, but at matmul time it **dequantizes them on the fly** back to bf16/fp32 before computing. The Apple Metal (MPS) path has no fused low-bit matmul kernel that consumes the low-bit representation directly. So memory bandwidth drops, but every forward pass now pays an extra dequantization cost.

That cost lands hardest on decode. Decoding passes the entire language model’s weights once per generated token, so across 128 generated tokens the dequantization overhead is billed 128 separate times. Prefill, by contrast, processes all 419 prompt tokens in a single batched pass, which amortizes that overhead far more effectively. TTFT in fact got shorter, not longer, going from bf16’s 2.94s to INT8’s 1.77s, consistent with the reduced-bandwidth benefit outweighing the dequantization cost at that batch size. But at INT4, TTFT swings back up to 9.91 seconds, slower than bf16. Four-bit packing is denser in storage but costlier to unpack and dequantize, and at some point that cost outweighs the bandwidth benefit and the balance flips. We did not profile exactly where that crossover happens as a function of batch size and sequence length; we name that gap explicitly in section 5.

The engineering lesson is clear. **Memory and speed are separate axes**. Low-bit weights do not, by themselves, guarantee speed. For edge quantization to actually pay off in latency, it needs a runtime with fused kernels that operate directly on the low-bit representation, MLX and the `llama.cpp`/GGUF family being the standard examples. In this experiment, the community MLX INT4 checkpoint was exactly that path, but it never got the chance to prove it, because the custom `mage_vl` architecture is not yet registered in `mlx-vlm` (section 3.1). Getting both “smaller and faster” at once requires not a weight-only path like `quanto`, but **a fused-kernel runtime with a completed architecture port**. That, backed by data, is this paper’s conclusion.

6 Limitations and scope

A few boundaries need stating clearly before generalizing this result.

The streaming-video path is out of scope. The DCVC-RT neural codec’s fast path is bound to CUDA .cu

extensions and does not run as shipped on Apple Metal. This is not a model defect; it is a hardware portability boundary, and we exclude it from this paper’s scope rather than hide it. Image/frame-level understanding never touches this codec and runs fully on Metal regardless of this measurement.

The MLX-native path needs an architecture port. Getting “small and fast” together on the same hardware requires porting the `mage_vl` architecture into `mlx-vlm`. This paper did not complete that port, so we could not measure the MLX path’s speed, only its 2.9GB on-disk size.

The quality check does not generalize. Our quality judgment is a qualitative comparison against ground truth on one synthetic image and one prompt. It is neither an automated score nor a statistical accuracy measure over many images and prompts. “All three precisions were accurate on this specific test” should not be generalized into “this model is always accurate under quantization.”

The authors’ reported numbers are a different condition entirely. The visual-token reduction, speedup, and Food-101/ImageNet/SoccerNet/ OVO-Bench figures cited in section 1.3 were measured by the authors on an 8xB200 datacenter node, under a hardware, dataset, and evaluation methodology that has nothing in common with our consumer-Mac measurement. We do not compare the two sets of numbers directly or use one as evidence for the other.

7 Data and reproducibility

- **Model:** `microsoft/Mage-VL` (Hugging Face), Apache-2.0 license. Mage-ViT visual encoder + Qwen3-4B-Instruct-2507 language backbone. Measured loaded parameter count: 4.742B. StreamMind event-gate weights: 1.07GB. DCVC-RT neural video codec (C++/CUDA extensions).
- **Hardware:** Apple M4 Pro, 48GB unified memory, macOS, arm64.
- **Software:** `transformers` 5.12, `torch` 2.12, device MPS. Quantization via `optimum-quanto` (weight-only, applied only to the `language_model` submodule; vision encoder stayed bf16). The MLX attempt used `mlx-vlm`.
- **Test input:** a synthetic 768x512 “station departures board” image, with ground truth we defined ourselves: title “CENTRAL STATION - DEPARTURES”, three rows (“14:05/PLATFORM 2/ON TIME”, “14:20/PLATFORM 5/DELAYED”, “14:35/PLATFORM 1/BOARDING”), a train pictogram, and a red STOP sign. Not a real photograph.
- **Prompt/generation settings:** a detailed-description request (“Describe this image in detail...”). Prompt length including vision tokens was 419 tokens; generation was fixed at 128 tokens.
- **Measurement method:** device memory is MPS-allocated memory, decode tok/s is averaged over the 128-token generation window, TTFT is the latency from end of 419-token prefill to the first output token. Quality is a qualitative comparison against ground truth.
- **Reproduction note:** the MLX 4-bit path does not load as-is on `mlx-vlm` mainline because the `mage_vl` architecture is not registered (“Model type `mage_vl` not supported”). Reproducing its speed requires completing that architecture port first.

8 References

1. Hugging Face model card: `microsoft/Mage-VL` (Apache-2.0), huggingface.co/microsoft/Mage-VL
2. Qwen/Qwen3-4B-Instruct-2507 model card, huggingface.co/Qwen
3. Hugging Face `transformers` library documentation, huggingface.co/docs/transformers
4. `optimum-quanto` (weight-only quantization), github.com/huggingface/optimum-quanto
5. `mlx-vlm` (Apple MLX vision-language model runtime), github.com/Blaizzy/mlx-vlm
6. Apache License 2.0, apache.org/licenses/LICENSE-2.0